10 Configuring an ASP.NET Core application

This chapter covers

- Loading settings from multiple configuration providers
- Storing sensitive settings safely
- Using strongly typed settings objects
- Using different settings in different hosting environments

In part 1 of this book, you learned the basics of getting an ASP.NET Core app up and running, and how to use minimal API endpoints to create an HTTP API. When you start building real applications, you'll quickly find that you want to tweak various settings at deploy time without necessarily having to recompile your application. This chapter looks at how you can achieve this task in ASP.NET Core by using configuration.

I know. Configuration sounds boring, right? But I have to confess that the configuration model is one of my favorite parts of ASP.NET Core; it's so easy to use and so much more elegant than some approaches in old versions of .NET Framework. In section 10.2 you'll learn how to load values from a plethora of sources—JavaScript Object Notation (JSON) files, environment variables, and command-line arguments—and combine them into a unified configuration object.

On top of that, ASP.NET Core makes it easy to bind this configuration to strongly typed options objects—simple plain old CLR object (POCO) classes that are populated from the configuration object, which you can inject into your services, as you'll see in section 10.3. Binding to strongly typed options objects lets you nicely encapsulate settings for different features in your app.

In the final section of this chapter, you'll learn about the ASP.NET Core hosting environments. You often want your app to run differently in different situations, such as running it on your developer machine compared with deploying it to a